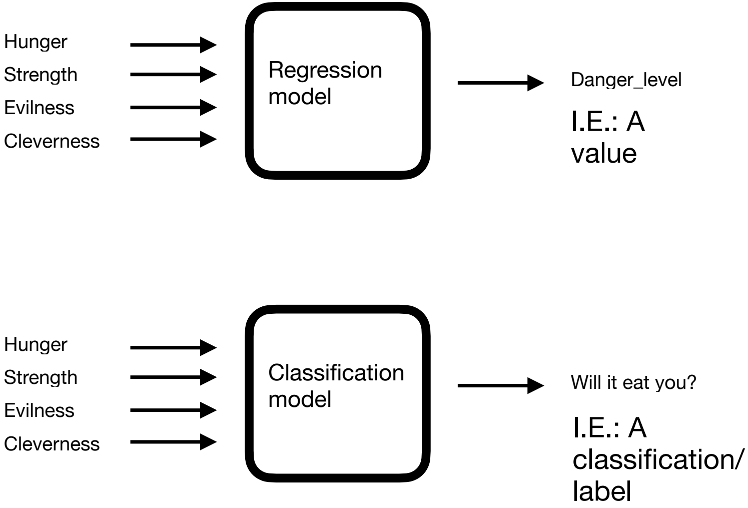
Sparse matrix example:

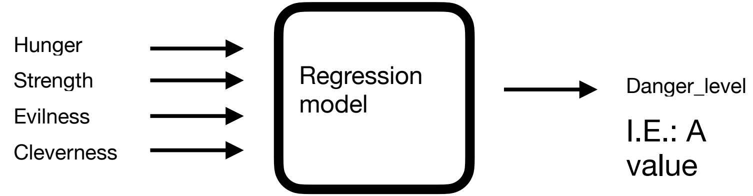
Extract features based on words,
construct a sparse matrix and
train a regression model

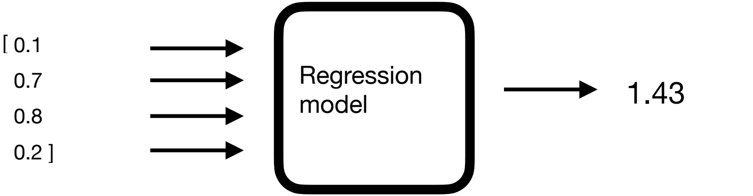
1. Regression and Classification

Difference between regression and classification

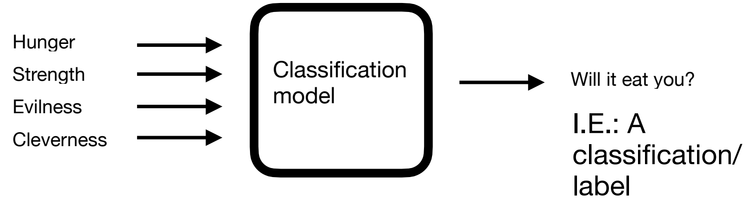


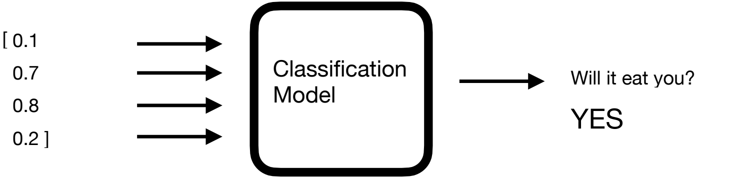
Regression predicts a value (e.g., the danger level), classification predicts a label (e.g. will it eat you? YES or NO.)





Regression predicts a value (e.g., the danger level).





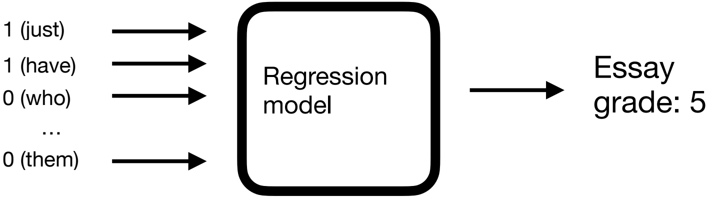
Classification predicts a label (e.g. will it eat you? YES or NO.)

2. Extract features based on words, and construct a sparse matrix

Data in a TSV-file:

essay_set	essay	rate	rater	rat	domain1_score
1	Dear local newspaper, I think effects computers have on people are great learning sk	4	4		8
1	Dear @CAPS1 @CAPS2, I believe that using computers will benefit us in many ways	5	4		9
1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more people use computers, but not ev	4	3		7
1	Dear Local Newspaper, @CAPS1 I have found that many experts say that computers	5	5		10
1	Dear @LOCATION1, I know having computers has a positive effect on people. The co	4	4		8
1	Dear @LOCATION1, I think that computers have a negative affect on us! How many p	4	4		8
1	Did you know that more and more people these days are depending on computers fo	5	5		10
1	@PERCENT1 of people agree that computers make life less complicated. I also agre	5	5		10
1	Dear reader, @ORGANIZATION1 has had a dramatic effect on human life. It has chan	4	5		9
1	In the @LOCATION1 we have the technology of a computer. Some say that the comp	5	4		9
1	Dear @LOCATION1, @CAPS1 people acknowledge the great advances that compute	4	4		8
1	Dear @CAPS1 @CAPS2 I feel that computers do take away from peoples life and are	4	4		8
1	Dear local newspaper I raed ur argument on the computers and I think they are a pos	4	3		7
1	My three detaileds for this news paper article is one state you opinion about the effec	3	3		6
1	Dear, In this world today we should have everyone useing computers. Computers ha	3	3		6
1	Dear @ORGANIZATION1, The computer blinked to life and an image of a blonde hair	6	6		12
1	Dear Local Newspaper, I belive that computers have a negative effect on peoples live	4	4		8

Task: Based on the words in the essay, predict the essay score.



Regression predicts a value, e.g. grade 5, or grade 4.5

```
"I think this is good stuff useing lots of these words"
--> Score: 4.89 (on a scale from 2.0 to 12.0)

"I believe life without internet
would be less productive"
--> Score: 5.12 (on a scale from 2.0 to 12.0)
```

The data is divided into different essay_sets:

2	I dont believe books should be takin off of the shelves for being offensive. Some boo	3	3		3
2	Many people go to libraries; such as children, parents, and grandparents. I have com	4	4		4
2	There are many people that think certain books, music, movies, and magazines are c	4	4		4
2	I think that if there are books, music @CAPS1, movies, or magazines that we should	3	3		3
2	Going into a @CAPS1 everyone has there own opinons as to what they should see c	3	3		3
2	All libraries are not good cause some people are going to library an getting on the co	2	2		2
2	There are more than six billion people in the world. That means there are over six billi	4	4		4
2	What is offensive to you? See your answer might be different from the person sitting	3	4		3
2	I, personally, do not think that books, music, movies, or magazines should be remove	4	4		4
2	Why @CAPS1 @CAPS2? Is there movies, music, magazines, books, and more in c	4	3		4
2	I believe censorship should be used in libraries to a certain extent. There are too mar	4	4		4
2	Have you ever been offended by a movie, book, song, or magazine? Did you have a	4	4		4
2	Libraries have tons of books from comedy all the way to violent.Some people like rea	4	4		4
2	Real life is rated @CAPS1 so why pretend it's not? If a library feels that if a book is to	4	4		4
2	There are many books in the library that shouldn't be there. From books about racisn	3	3		3
2	Books can be good things, but are they always good? They teach us things, entertai	4	4		4
2	Just because I agree with something, do you have to agree too ? Your cousin believi	4	4		4
2	offensive materials in libraries should not be taken off of the shelves because of their	4	3		4

Example essay

```
Dear local newspaper, I think effects computers have
on people are great learning skills/affects because
they give us time to chat with friends/new people,
helps us learn about the globe(astronomy) and keeps us
out of troble! Thing about! Dont you think so? How
would you feel if your teenager is always on the phone
with friends! Do you ever time to chat with your
friends or buisness partner about things. Well now -
there's a new way to chat the computer, theirs plenty
of sites on the internet to do so: @ORGANIZATION1,
@ORGANIZATION2, @CAPS1, facebook, myspace ect. Just
think now while your setting up meeting with your boss
on the computer, your teenager is having fun on the
phone not rushing to get off cause you want to use it.
How did you learn about other countrys/states outside
of yours? Well I have by computer/internet, it's a
new way to learn about what going on in our time! You
might think your child spends a lot of time on the
```

Relevant columns: essay_set, essay, domain1_score

essay_set	essay	rate	rater	rat	domain1_score
1	Dear local newspaper, I think effects computers have on people are great learning sk	4	4		8
1	Dear @CAPS1 @CAPS2, I believe that using computers will benefit us in many ways	5	4		9
1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more people use computers, but not ev	4	3		7
1	Dear Local Newspaper, @CAPS1 I have found that many experts say that computers	5	5		10
1	Dear @LOCATION1, I know having computers has a positive effect on people. The co	4	4		8
1	Dear @LOCATION1, I think that computers have a negative affect on us! How many p	4	4		8
1	Did you know that more and more people these days are depending on computers fo	5	5		10
1	@PERCENT1 of people agree that computers make life less complicated. I also agre	5	5		10
1	Dear reader, @ORGANIZATION1 has had a dramatic effect on human life. It has chan	4	5		9
1	In the @LOCATION1 we have the technology of a computer. Some say that the comp	5	4		9
1	Dear @LOCATION1, @CAPS1 people acknowledge the great advances that compute	4	4		8
1	Dear @CAPS1 @CAPS2 I feel that computers do take away from peoples life and are	4	4		8
1	Dear local newspaper I raed ur argument on the computers and I think they are a pos	4	3		7
1	My three detaileds for this news paper article is one state you opinion about the effec	3	3		6
1	Dear, In this world today we should have everyone useing computers. Computers ha	3	3		6
1	Dear @ORGANIZATION1, The computer blinked to life and an image of a blonde hair	6	6		12
1	Dear Local Newspaper, I belive that computers have a negative effect on peoples live	4	4		8

Convenient Python module: **csv**, for reading and writing data in table format, e.g. data separated by tab or comma.
(Compare with the json module, for reading an writing json format.)

`csv` — CSV File Reading and Writing

Source code: Lib/csv.py

The so-called CSV (Comma Separated Values) format is the most common import and export format for spreadsheets and databases. CSV format was used for many years prior to attempts to describe the format in a standardized way in **RFC 4180**. The lack of a well-defined standard means that subtle differences often exist in the data produced and consumed by different applications. These differences can make it annoying to process CSV files from multiple sources. Still, while the delimiters and quoting characters vary, the overall format is similar enough that it is possible to write a single module which can efficiently manipulate such data, hiding the details of reading and writing the data from the programmer.

The `csv` module implements classes to read and write tabular data in CSV format. It allows programmers to say, “write this data in the format preferred by Excel,” or “read data from this file which was generated by Excel,” without knowing the precise details of the CSV format used by Excel. Programmers can also describe the CSV formats understood by other applications or define their own special-purpose CSV formats.

The `csv` module’s `reader` and `writer` objects read and write sequences. Programmers can also read and write data in dictionary form using the `DictReader` and `DictWriter` classes.

`https://docs.python.org/3/library/csv.html`

Read data, using csv.DictReader

```python
def read_essay_sets(filename: str) -> dict[int, list[dict]]:
    """Read all essay sets from an ASAP data file.
    Args:
        filename -- tab-separated ASAP data file,
                    cleaned to be unicode compatible
    Returns:
        dict mapping essay set number (int) to a list
        of dicts, representing
        all the essays in the given essay set.
    """
    essay_set = defaultdict(list)
    with open(filename, "r", newline="") as f:
        reader = csv.DictReader(f, delimiter="\t")
        for row in reader:
            essay_set[int(row["essay_set"])].append(
                        {"text": row["essay"],
                         "score": int(row["domain1_score"])})
    return essay_set
```

The defaultdict:

```python
from collections import defaultdict

my_dict = defaultdict(list)

my_dict["hot_drinks"].append("tea")
print(1, my_dict)
my_dict["hot_drinks"].append("coffee")
print(2, my_dict)
print(3, my_dict["cold_drinks"])
print(4, my_dict)
```

```
1 defaultdict(<class ’list’>, {’hot_drinks’: [’tea’]})

2 defaultdict(<class ’list’>,
{’hot_drinks’: [’tea’, ’coffee’]})

3 []

4 defaultdict(<class ’list’>,
{’hot_drinks’: [’tea’, ’coffee’], ’cold_drinks’: []})
```

Extract a list of words from the text

```
def extract_features(text: str) -> list[str]:
    """Extract features from a text.

    Currently, we simply extract all words,
    ignoring the anonymized
    placeholders of the form @SOMETHING123.

    Args:
        text -- essay text

    Returns:
        list of feature names
    """
    return re.findall(r"[\w']+",\
        re.sub(r'@[A-Z]+\d+', '', text).lower())
```

```
['dear', 'local', 'newspaper', 'i',
'think', 'effects', 'computers',
'have', 'on', 'people',
....
```

- List comprehension in list comprehension (or rather, generator expression in generator expression).
- Dict comprehension

```
def construct_matrix(essays: list[dict]) -> tuple:
    essay_features = [set(extract_features(essay["text"]))
                      for essay in essays]

    # Create an ordered list of all features that occur
    # at least in 5 different essays.
    feature_counts = Counter(feature
                             for features in essay_features
                             for feature in features)
    features = [feature
                for feature, count in feature_counts.items()
                if count >= 5]
    feature_idx = \
    {feature: i for i, feature in enumerate(features)}
```

Each word that will be used as a feature receives a column index

```
{'just': 0, 'have': 1, 'buisness': 2, 'feel': 3,
'day': 4, 'time': 5, 'or': 6, 'meeting': 7,
'hope': 8, 'other': 9, 'theirs': 10, 'doing': 11,
'spends': 12, 'in': 13, 'of': 14, 'thing': 15,
'learning': 16, 'but': 17, 'drive': 18, 'lot': 19,
'agree': 20, 'to': 21, 'much': 22, 'teenager': 23,
'us': 24, 'off': 25, 'fun': 26, 'safe': 27,
'by': 28, 'way': 29, 'setting': 30, 'interesting': 31,
...
```

This index is used for the column index when creating the sparse matrix.

```
{'just': 0, 'have': 1, 'buisness': 2, 'feel': 3,
'day': 4, 'time': 5, 'or': 6 ...
```

This is what the matrix in standard-format would look like:

	0	1	2	3	4	5	6	...	158	...	272	...	330	...
	just	have	buisness	feel	day	time	or	...	them	...	who	...	times	...
Essay 1	1	1	1	1	1	1	1	...	0	...	0	...	1	...
Essay 2	0	0	1	0	0	0	1	...	1	...	0	...	1	...
Essay 3	0	0	0	0	0	0	1		0		1	...	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Essay 1783	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Would take up a lot of space filled as a standard Numpy matrix:

```
[[1. 1. 1. ... 0. 0. 0.]
[0. 0. 1. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
...
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]]
```

```python
# Construct the matrix as a list of coordinates
# whose values will be 1.0
coord_i = []
coord_j = []
for i, current_features in enumerate(essay_features):
    for feature in current_features:
        j = feature_idx.get(feature)
        if j is not None:
            coord_i.append(i) # append essay-index
            coord_j.append(j) # append feature-index

# Create a SciPy sparse matrix given the
# coordinates above
X = scipy.sparse.\
    csr_array((np.ones_like(coord_i), (coord_i, coord_j)),
            shape=(len(essays), len(features)),
            dtype=np.float64)
```

`np.ones_like` creates an array of the same shape and type as the array given as argument.

Elements are added to the array representing filled rows (essays), and the array representing filled columns (word features)

```
...
For essay nr 2, add the word 'times' as feature
'times' has index 330
to the array for filled rows, append: 2
to the array for filled columns, append: 330
-----
coord_i (rows): [... 2]
coord_j (columns): [... 330]
```

	0	1	2	3	4	5	6	...	158	...	272	...	330	...
	just	have	buisness	feel	day	time	or	...	them	...	who	...	times	...
Essay 1	1	1	1	1	1	1	1	...	0	...	0	...	1	...
Essay 2	0	0	1	0	0	0	1	...	1	...	0	...	1	...
Essay 3	0	0	0	0	0	0	1		0		1	...	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Essay 1783	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Elements are added to the array representing filled rows (essays), and the array representing filled columns (word features)

```
...
For essay nr 2, add the word 'them' as feature
'them' has index 158
to the array for filled rows, append: 2
to the array for filled columns, append: 158
-----
coord_i (rows): [... 2, 2]
coord_j (columns): [... 330, 158]
```

	0	1	2	3	4	5	6	...	158	...	272	...	330	...
	just	have	buisness	feel	day	time	or	...	them	...	who	...	times	...
Essay 1	1	1	1	1	1	1	1	...	0	...	0	...	1	...
Essay 2	0	0	1	0	0	0	1	...	1	...	0	...	1	...
Essay 3	0	0	0	0	0	0	1		0		1	...	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Essay 1783	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Elements are added to the array representing filled rows (essays), and the array representing filled columns (word features)

```
...
For essay nr 3, add the word 'who' as feature
'who' has index 272
to the array for filled rows, append: 3
to the array for filled columns, append: 272
...
coord_i (rows): [... 2, 2, 3]
coord_j (columns): [... 330, 158, 272]
```

	0	1	2	3	4	5	6	...	158	...	272	...	330	...
	just	**have**	**buisness**	**feel**	**day**	**time**	**or**	**...**	**them**	**...**	**who**	**...**	**times**	**...**
Essay 1	1	1	1	1	1	1	1	...	0	...	0	...	1	...
Essay 2	0	0	1	0	0	0	1	...	1	...	0	...	1	...
Essay 3	0	0	0	0	0	0	1		0		1	...	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Essay 1783	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Elements are added to the array representing filled rows (essays), and to the array representing filled columns (word features)

```
coord_i, essays [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 3 ...]
coord_j, feature-words [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91,
92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112,
113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131,
132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150,
151, 152, 153, 154, 155, 156, 157, 158, 1, 3, 159, 160, 161, 162, 163, 164, 165, 166, 6, 8, 9,
10, 167, 168, 169, 170, 12, 171, 13, 18, 172, 173, 174, 175, 176, 177, 21, 22, 24, 178, 179,
180, 181, 29, 30, 32, 182, 35, 183, 40, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 42,
194, 45, 47, 50, 51, 195, 196, 197, 198, 199, 200, 201, 202, 63, 65, 203, 66, 67, 204, 205, 206,
207, 208, 70, 71, 209, 72, 210, 211, 212, 213, 214, 75, 215, 216, 82, 217, 84, 218, 219, 86,
220, 221, 222, 223, 89, 90, 224, 92, 93, 94, 225, 226, 98, 227, 228, 229, 230, 231, 232, 102,
233, 234, 106, 235, 236, 237, 116, 238, 239, 115, 118, 240, 121, 241, 242, 243, 123, 124, 244,
245, 246, 129, 130, 247, 248, 249, 133, 250, 251, 252, 253, 254, 255, 137, 140, 256, 257, 145,
258, 146, 259, 260, 148, 261, 262, 263, 264, 265, 266, 152, 267, 268, 150, 153, 269, 270, 271,
272, 1, 2, 3, 161, 273, 274, 275, 166, 7, 8, 6, 276, 277, 278, 12, 13, 171, 18, 279, 280, 281,
22, 23, 24, 282, 283, 180, 29, 30, 32, 284, 285, 286, 182, 36, 183, 287, 40, 288, 289, 45, 46 ...]
```

The following sparse matrix is created:

```
<Compressed Sparse Row sparse array of dtype 'float64'
 with 272199 stored elements and shape (1783, 3503)>

  Coords Values
  (0, 0) 1.0
  (0, 1) 1.0
  (0, 2) 1.0
  : :
  (1782, 3212) 1.0
  (1782, 3252) 1.0
  (1782, 3502) 1.0
```

Instead of a very large matrix with the standard Numpy format, containing mostly zeros:

```
[[1. 1. 1. ... 0. 0. 0.]
[0. 0. 1. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
...
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]]
```

```python
def construct_matrix(essays: list[dict]) -> tuple:
    ...

    # Create a SciPy sparse matrix
    X = scipy.sparse.\
        csr_array((np.ones_like(coord_i), (coord_i, coord_j)),
                  shape=(len(essays), len(features)),
                  dtype=np.float64)
    # The target values are dense,
    # so we use the essay scores directly
    y = np.array([essay["score"] for essay in essays],\
        dtype=np.float64)

    return X, y, features, feature_idx
```

We now have the data in a format we can use to train our model:

```python
def main(filename):
    # Read the ASAP data (all sets)
    essay_sets = read_essay_sets(filename)
    # Construct matrices from essay set 1 only
    X, y, features, feature_idx = \
    construct_matrix(essay_sets[1])
```

3. Train and apply a regression model

```python
clf = Ridge(alpha=1e3)
clf.fit(X, y)

def grade_essays(texts):
    # Create a feature matrix containing the given texts
    X_test = scipy.sparse.dok_array(
              (len(texts), len(features)),\
               dtype=np.float64)

    for i, text in enumerate(texts):
        for feature in extract_features(text):
            j = feature_idx.get(feature)
            if j is not None:
                X_test[i, j] = 1.0
    y_test = clf.predict(X_test)
    return y_test

examples = ["I think this is good stuff ..",
            "I believe life without ..."]
print(grade_essays(examples))
```

```
[4.88992927 5.12142997]
```